Quiz 11-1

- B-tree can return early at an internal node; B+-tree always descends to a leaf.
- So why does virtually every database use B+-trees?

힌트 보기

개념 정리 · B+-tree의 구조

- 실데이터(레코드 포인터)는 **leaf에만** 두고 내부 노드는 라우팅용 키만 담는다. 모든 leaf는 **연결리스트**로 이어져 있다.
- 데이터베이스의 비용 단위는 메모리 비교가 아니라 **디스크 I/O**다. 내부 노드에 키가 더 많이 들어가면 fan-out이 커진다.

강의 슬라이드 Ch11 p09 Quiz

정답 보기

풀이

① **낮은 트리 높이 → 디스크 I/O 감소.** 내부 노드가 데이터 대신 키만 담으므로 한 노드에 키를 더 많이 넣을 수 있어 fan-out이 커지고 높이가 낮아진다.

> 트리 높이가 낮아지면 루트에서 leaf까지 방문하는 노드 수가 줄어들어 디스크 I/O도 감소할 수 있다. 노드 하나가 디스크 페이지 하나에 대응하고 캐시 효과를 무시하면, 탐색에 필요한 페이지 접근 횟수는 방문한 노드 수에 비례한다. 실제 I/O 횟수는 캐시 상태 등에 따라 달라지므로 트리 높이와 항상 같지는 않다.

② **효율적인 range query와 순차 스캔.** 모든 값이 leaf에 정렬되어 있고 leaf끼리 연결리스트로 이어져 있어 범위 탐색이 시작점 탐색 후 순차 스캔 $O(k)$로 끝난다.

> B-tree에서도 중위 순회를 이용해 범위 검색을 수행할 수 있다. 다만 데이터가 내부 노드와 leaf에 나뉘어 저장될 수 있어, 연결된 leaf를 순서대로 따라가는 B+-tree보다 순차 스캔 시 페이지 접근이 복잡해질 수 있다. B+-tree는 시작 키를 찾은 뒤 정렬된 leaf를 연속적으로 탐색할 수 있어 대량의 범위 검색에 유리하다.

③ **균일하고 예측 가능한 탐색 깊이.** B+-tree는 모든 키의 검색 경로가 leaf까지 이어지므로 인덱스 탐색 깊이가 균일하다. B-tree는 내부 노드에서 키를 찾으면 더 일찍 종료할 수 있지만, B+-tree는 높은

fan-out과 연결된 leaf 구조를 통해 디스크 기반 탐색 및 범위 검색에서 장점을 갖는다. 실제 성능 차이는 데이터 분포, 조회 패턴, 캐시와 구현 방식에 따라 달라질 수 있다.

최종 답 **fan-out↑(높이↓·I/O↓) + leaf 연결로 range query 효율 + 균일한 탐색 깊이**

한 줄 정리 · B-tree는 내부 노드에서 조기 종료할 수 있지만, B+-tree는 높은 fan-out에 따른 낮은 높이, 연결된 leaf를 통한 효율적인 범위 스캔, 균일한 탐색 깊이 덕분에 데이터베이스 인덱스에 널리 사용된다.